

Arise Lead Engineer Assessment Q1–Q136

Correct answers are green with ✓ after the correct option.

Question 1

Question: A junior developer uses a highly descriptive variable name like ``customerOrderProcessorWithDiscountAndTaxCalculationStrategy``. What 'clean code' principle is MOST likely being over-applied or misunderstood, and what is the typical underlying issue it masks?

Choices:

- A. Principle: Avoid Duplication (DRY). Issue: Lack of appropriate abstraction.
- B. Principle: Prefer Polymorphism over If/Else. Issue: Over-reliance on conditional logic.
- C. Principle: Keep Functions Small. Issue: Deeply nested code structures.

D. Principle: Intention-Revealing Names. Issue: Violation of the Single Responsibility Principle (SRP). ✓

Explanation: A very long variable name may reveal that one variable or object is carrying too many responsibilities. The issue is not naming itself, but that the design likely violates SRP and should be decomposed into clearer abstractions.

Question 2

Question: Which design pattern is used to manage multiple dependencies between objects?

Choices:

- A. Factory Pattern
- B. Decorator Pattern
- C. Observer Pattern ✓**
- D. Singleton Pattern

Explanation: Observer is designed for one-to-many dependency management, where multiple subscribers are notified when a subject changes.

Question 3

Question: Which of the following are best practices when working with secrets in production environments?

Choices:

- A. Store secrets in environment variables ✓**
- B. Use secret management tools or vaults ✓**
- C. Commit secrets to git in encrypted form

D. Hardcode secrets into source code for fast access

Explanation: Secrets should be externalized from source code and managed through environment configuration or a dedicated secrets vault to reduce leakage risk and support rotation.

Question 4

Question: Which practice most effectively increases code maintainability in a growing codebase?

Choices:

A. Applying the Single Responsibility Principle consistently ✓

B. Writing long methods to reduce function calls

C. Using global variables for shared state

D. Favoring inheritance over composition

Explanation: SRP keeps modules focused and easier to change, test, and understand as the codebase grows.

Question 5

Question: Which of the following is a benefit of using version control systems in software development?

Choices:

A. They allow multiple developers to work on different parts of the project simultaneously without overwriting each other's changes. ✓

B. They provide a platform for real-time communication between developers working on different projects.

C. They automatically fix bugs in the code when new updates are pushed.

D. They replace the need for testing by ensuring code integrity through automated checks.

Explanation: Version control enables parallel collaboration, history tracking, branching, and merging without developers overwriting one another's work.

Question 6

Question: Which practice most effectively prevents merge conflicts in a collaborative Git workflow?

Choices:

A. Never reviewing pull requests

B. Using only force pushes to update branches

C. Committing large, infrequent changes

D. Regularly pulling and rebasing from the main branch before pushing changes ✓

Explanation: Regularly synchronizing with the main branch reduces divergence and exposes conflicts early while they are easier to resolve.

Question 7

Question: What is an API, and how does it facilitate communication between different software systems?

Choices:

- A. APIs allow users to manually input data into different systems without any automated processes.
- B. An API is a programming language used to write applications that can be executed on different platforms.
- C. APIs are primarily used for encrypting data between two parties in communication.

D. An API is a set of rules that allows one software application to interact with another by defining the methods and data formats for exchanging information. ✓

Explanation: An API defines the contract, methods, and data formats that let applications exchange information programmatically.

Question 8

Question: What is the primary benefit of using a microservice architecture in software development?

Choices:

- A. Reduced cost of ownership
- B. Easier maintenance and updates

C. Improved scalability ✓

- D. Faster deployment times

Explanation: Microservices allow individual services to scale independently according to workload, improving overall scalability.

Question 9

Question: When a junior developer submits a pull request with a commit message stating 'Fix bug', a senior engineer requests a more descriptive message. Which of the following is the MOST compelling reason for this insistence, from a 'Git effective using' perspective?

Choices:

- A. To enable future 'git bisect' operations to efficiently pinpoint the introduction of regressions.
- B. To ensure the commit history can be easily used for 'git blame' to identify the author of the change.
- C. To simplify the process of squashing commits into a single logical change before merging.

D. To facilitate automated release notes generation and changelog creation. ✓

Explanation: Clear commit messages make change history useful for automation, release notes, and future maintenance.

Question 10

Question: What is the main purpose of the 'try-catch' block in exception handling?

Choices:

A. To repeat a block of code until it executes successfully.

B. To handle exceptions that occur during the execution of a program. ✓

C. To execute a block of code that may cause an error.

D. To debug a program by logging errors to a file.

Explanation: try-catch separates normal logic from error-handling logic and allows programs to respond to runtime exceptions safely.

Question 11

Question: Which actions help improve database query performance?

Choices:

A. Adding appropriate indexes to frequently queried columns ✓

B. Storing all computed results in a single table

C. Using nested subqueries without analyzing execution plan

D. Avoiding SELECT * in production queries ✓

Explanation: Indexes reduce search cost for frequent predicates, and avoiding SELECT * reduces unnecessary I/O and data transfer.

Question 12

Question: You're testing a function that calls a third-party API with unstable uptime. What is the most robust testing strategy?

Choices:

A. Call the real API and retry failed tests later

B. Use a mock API that simulates all edge cases and failures ✓

C. Disable these tests and focus on unit tests only

D. Use the real API and mark tests as optional

Explanation: Mocking unstable dependencies makes tests deterministic while still validating expected success and failure behavior.

Question 13

Question: Which practices help prevent flaky tests in automated pipelines?

Choices:

A. Avoid tests that assert logs and metrics

B. Use randomly generated test data for every run

C. Avoid using hardcoded sleep() in tests ✓

D. Mock or stub unstable dependencies ✓

Explanation: Hardcoded waits create timing fragility, while stubbing unstable dependencies improves repeatability.

Question 14

Question: Which of the following best describes mutation testing in the context of unit testing?

Choices:

A. It validates if all branches in code are executed at least once.

B. It measures the time taken for tests to execute after code changes.

C. It involves introducing small changes to code to check if existing tests detect the errors. ✓

D. It checks if code coverage increases after each commit.

Explanation: Mutation testing checks whether tests can detect small intentional faults, measuring assertion effectiveness rather than only execution coverage.

Question 15

Question: Which of the following is the most subtle risk of using random data generation in property-based testing?

Choices:

A. Random data always makes tests deterministic.

B. Random data always increases test speed.

C. Rare edge cases may still be missed if not enough iterations are run. ✓

D. Random data generation eliminates the need for assertions.

Explanation: Random generation improves coverage breadth but cannot guarantee that rare edge cases are sampled without enough iterations or targeted generators.

Question 16

Question: In a CI pipeline, a test job sometimes hangs and never completes. What should you do?

Choices:

- A. Run the job with lower CPU priority
- B. Switch to manual regression testing
- C. Add a timeout to the test job execution ✓**
- D. Schedule the job outside of working hours

Explanation: Timeouts prevent stuck jobs from blocking the pipeline indefinitely and make failures visible.

Question 17

Question: Which of the following is the most subtle risk of using code coverage as a gate for merging code?

Choices:

- A. Coverage gates eliminate the need for code reviews.
- B. Coverage gates guarantee zero bugs.
- C. Coverage gates always improve test quality.
- D. Developers may write superficial tests to satisfy coverage requirements. ✓**

Explanation: Coverage measures executed lines, not the quality of assertions; teams may game the metric with weak tests.

Question 18

Question: In TDD, which of the following is the most accurate description of the 'Red' phase?

Choices:

- A. Writing code to make the test pass.
- B. Running all tests to ensure none fail.
- C. Writing a test that fails because the required functionality is not yet implemented. ✓**
- D. Refactoring code to improve design without changing behavior.

Explanation: The Red phase proves the test is meaningful by first observing it fail before implementation.

Question 19

Question: Which of the following is the most subtle risk of using only static analysis tools for software quality?

Choices:

- A. Static analysis always slows down development.
- B. They may miss runtime errors that only occur in specific environments. ✓**

C. Static analysis eliminates the need for documentation.

D. Static analysis always guarantees bug-free code.

Explanation: Static analysis examines code without executing it, so environment-specific and runtime behavior can still be missed.

Question 20

Question: For a banking system that needs to operate across multiple countries with different regulatory requirements, which architecture best supports regional customization while maintaining a core codebase?

Choices:

A. Microservices with bounded contexts

B. Microkernel architecture ✓

C. Pipeline architecture

D. Space-based architecture

Explanation: Microkernel architecture keeps a stable core while allowing region-specific plug-ins or extensions.

Question 21

Question: Which architectural style is most appropriate for a system that processes large volumes of financial transactions with eventual consistency requirements?

Choices:

A. Event-driven architecture ✓

B. Model-View-Controller pattern

C. Monolithic architecture

D. Layered architecture

Explanation: Event-driven architecture supports asynchronous processing, decoupling, and eventual consistency across services.

Question 22

Question: When monitoring a critical financial application across multiple cloud regions, which metric would provide the most accurate insight into global system health?

Choices:

- A. Database Performance
- B. Error Rate
- C. Response Time
- D. Network Latency
- E. Transaction Volume
- F. Memory Utilization
- G. Security Events
- H. CPU Utilization
- I. Storage Utilization
- J. Service Availability

Explanation: Service availability directly reflects whether the application is reachable and functioning across regions.

Question 23

Question: In the context of financial trading platforms, which pattern best supports high-frequency trading operations?

Choices:

- A. Throttling pattern
- B. Saga pattern

C. CQRS pattern with event sourcing ✓

D. Circuit Breaker pattern

Explanation: CQRS separates reads and writes, while event sourcing captures ordered state changes, supporting performance and auditability.

Question 24

Question: For a financial ledger system that requires absolute data integrity and ACID compliance, which storage solution is most appropriate?

Choices:

A. Distributed file system

B. Relational database with appropriate isolation levels ✓

C. In-memory data grid

D. NoSQL database

Explanation: Relational databases with suitable isolation levels are designed for ACID transactions and strong consistency.

Question 25

Question: Which of the following techniques are appropriate for securing sensitive financial data at rest? (Select all that apply)

Choices:

A. Hardware Security Module Integration ✓

B. Data Masking

C. Transparent Data Encryption ✓

D. Transport Layer Security

E. Role-Based Access Control

F. Column-level Encryption ✓

Explanation: HSMs protect keys, TDE encrypts database files, and column-level encryption protects especially sensitive fields at rest.

Question 26

Question: During a post-mortem analysis following a major incident in a payment processing system, which maintenance strategy would be most effective for preventing similar incidents in the future?

Choices:

A. Increasing the frequency of scheduled maintenance windows

B. Conducting regular chaos engineering experiments ✓

C. Adding redundant infrastructure components

D. Implementing additional unit tests covering the failure scenario

Explanation: Chaos engineering validates system resilience by intentionally testing failure scenarios before real incidents occur.

Question 27

Question: For a financial organization implementing DevOps practices, which monitoring approach would best support the goal of reducing mean time to recovery (MTTR)?

Choices:

A. Alert aggregation with priority-based routing

B. Comprehensive dashboards with system health visualization

C. Automated incident detection with contextual alerting ✓

D. Post-incident analysis with root cause identification

Explanation: Automated contextual alerts reduce diagnosis time and route incidents faster, directly reducing MTTR.

Question 28

Question: Which approach would be most effective for monitoring and maintaining a financial application with strict regulatory compliance requirements?

Choices:

A. Pre-deployment compliance checks with validation gates

B. Separation of compliance monitoring from operational monitoring

C. Regular compliance audits with manual remediation tracking

D. Continuous compliance verification with automated evidence collection ✓

Explanation: Continuous evidence collection supports ongoing auditability and faster detection of compliance drift.

Question 29

Question: For a financial organization implementing site reliability engineering (SRE) practices, which approach to maintenance would align best with SRE principles?

Choices:

A. Toil reduction through automation and self-healing systems ✓

B. Centralized operations team with specialized expertise

C. Comprehensive preventative maintenance schedules

D. Change approval processes with risk assessment

Explanation: SRE emphasizes reducing manual toil through automation, reliability engineering, and self-healing systems.

Question 30

Question: Which of the following index types would be most effective for optimizing range queries on a column with high cardinality in a large database table?

Choices:

A. Full-text index

B. B+ tree index ✓

C. Bitmap index

D. Hash index

Explanation: B+ tree indexes preserve sorted order, making them effective for range scans and high-cardinality predicates.

Question 31

Question: Which factors contribute to the effectiveness of a composite index in a relational database?

Choices:

A. The cardinality of the indexed columns ✓

B. The use of a clustered index

C. The size of the database buffer cache

D. The order of columns in the index ✓

Explanation: Composite index usefulness depends heavily on selectivity/cardinality and matching the leading column order used by queries.

Question 32

Question: A database transaction fails due to a deadlock. Which isolation level is most likely to reduce the likelihood of deadlocks in a high-concurrency environment?

Choices:

A. Snapshot Isolation ✓

B. Read Committed

C. Serializable

D. Read Uncommitted

Explanation: Snapshot isolation reduces read/write blocking by using row versions, which can reduce deadlock likelihood.

Question 33

Question: Arrange the steps to configure a Redis Cluster for high availability in the correct order.

Choices:

A. Verify cluster status and test failover

B. Add replica nodes for each primary node

C. Install Redis on all nodes and enable cluster mode

D. Use redis-cli to create the cluster with node assignments

Correct order:

1. C. Install Redis on all nodes and enable cluster mode ✓

2. D. Use redis-cli to create the cluster with node assignments ✓

3. B. Add replica nodes for each primary node ✓

4. A. Verify cluster status and test failover ✓

Explanation: The cluster must first be installed and created, then replicas are assigned, and finally status/failover are verified.

Question 34

Question: When implementing a Redis pub/sub system for real-time notifications, what is the primary limitation that could impact scalability?

Choices:

A. Pub/sub requires Lua scripting for message delivery

B. Pub/sub messages are not persisted ✓

C. Pub/sub cannot be used in Redis Cluster

D. Pub/sub channels are limited to 1000 per instance

Explanation: Redis pub/sub is ephemeral; messages are not stored for offline or slow consumers.

Question 35

Question: Which docker-compose.yml configurations improve service reliability?

Choices:

- A. Set `network_mode: host` for all services
- B. Use `depends_on` to ensure service readiness

C. Define healthchecks to monitor service status ✓

D. Use `restart: always` to automatically restart failed containers ✓

Explanation: Healthchecks detect unhealthy services, and restart policies improve recovery from container failure.

Question 36

Question: A Dockerfile uses `COPY ./app` to copy files. What potential issue arises if the `.dockerignore` file is not properly configured?

Choices:

A. Unnecessary files like `.git` or `node_modules` are copied, increasing image size ✓

- B. The build fails due to missing files
- C. The build cache is invalidated
- D. The container cannot access the copied files

Explanation: Without `.dockerignore`, unnecessary local files enter the build context and image, increasing size and risk.

Question 37

Question: Arrange the steps to create, optimize, and run a secure Docker container from a Dockerfile for a production-ready application.

Choices:

- A. Test the image locally with `docker run --rm -it myapp:1.0`
- B. Build the Docker image with a tag using `docker build -t myapp:1.0 .`
- C. Write a Dockerfile with a specific base image (e.g., `FROM python:3.9-slim`)
- D. Push the image to a registry using `docker push myapp:1.0`
- E. Create a `.dockerignore` file to exclude unnecessary files

Correct order:

1. C. Write a Dockerfile with a specific base image (e.g., `FROM python:3.9-slim`) ✓

2. E. Create a `.dockerignore` file to exclude unnecessary files ✓

3. B. Build the Docker image with a tag using `docker build -t myapp:1.0`. ✓

4. A. Test the image locally with `docker run --rm -it myapp:1.0` ✓

5. D. Push the image to a registry using `docker push myapp:1.0` ✓

Explanation: A production image flow starts with the Dockerfile and context hygiene, then build, local test, and registry push.

Question 38

Question: What happens if you specify `CMD ["executable", "param1", "param2"]` in a Dockerfile without an `ENTRYPOINT`?

Choices:

A. The command is ignored, and the container runs the default shell

B. The container fails to start due to missing `ENTRYPOINT`

C. The command runs inside a shell like `/bin/sh -c`

D. The command runs as is, without a shell, when the container starts ✓

Explanation: Exec-form CMD runs directly as the container command without shell expansion.

Question 39

Question: Which of the following configurations are crucial for enabling an idempotent Kafka producer? (Select all that apply)

Choices:

A. `enable.idempotence=true` ✓

B. `transactional.id=some-id`

C. `retries > 0` ✓

D. `acks=all` ✓

Explanation: Kafka idempotent producers require idempotence enabled with retries and all acknowledgements to avoid duplicate writes.

Question 40

Question: In a Kafka Streams application, what is the primary difference between a `KStream` and a `KTable`?

Choices:

A. `KStream` represents an append-only stream of records, while `KTable` represents a changelog stream where each key is associated with the latest value. ✓

B. Joining a `KStream` with a `KTable` is not possible.

C. `KStream` can only be read from a single topic, while `KTable` can be materialized from multiple topics.

D. `KTable` operations are always windowed, while `KStream` operations are not.

Explanation: KStream models each record as an event, while KTable models the latest value per key.

Question 41

Question: A Kafka consumer is configured with `enable.auto.commit=false`. Under which circumstance could this lead to message reprocessing?

Choices:

A. The consumer application commits offsets before processing the messages and then crashes.

B. The consumer application crashes after processing a batch of messages but before manually committing the offsets. ✓

C. The consumer is part of a consumer group, and a rebalance occurs.

D. The broker that the consumer is fetching from goes down.

Explanation: If processing succeeds but offsets are not committed before a crash, Kafka will redeliver the same records.

Question 42

Question: A Kafka consumer is experiencing frequent rebalances. Which of the following could be potential causes? (Select all that apply)

Choices:

A. The value of `heartbeat.interval.ms` is set higher than `session.timeout.ms`. ✓

B. The consumer application is experiencing long garbage collection pauses. ✓

C. The value of `session.timeout.ms` is too low for the processing time. ✓

D. The `Workspace.min.bytes` setting is too high.

Explanation: Heartbeat/session misconfiguration or long pauses can make brokers think consumers are dead, triggering rebalances.

Question 43

Question: Which configurations are necessary to enable communication between two VPCs in different AWS regions?

Choices:

A. Public IP assignment to all instances

B. Route table updates to include remote VPC CIDR blocks ✓

C. AWS Transit Gateway with inter-region peering

D. VPC peering connection ✓

Explanation: Inter-region VPC communication needs a connectivity construct such as peering and routes to remote CIDR blocks.

Question 44

Question: Arrange the steps to configure a load balancer to route traffic to a new target group in the correct order.

Choices:

A. Configure health checks for the target group

B. Update the load balancer listener to route traffic to the target group

C. Create a target group and register instances

Correct order:

1. C. Create a target group and register instances ✓

2. A. Configure health checks for the target group ✓

3. B. Update the load balancer listener to route traffic to the target group ✓

Explanation: Targets must exist first, health checks should validate them, then listeners can route traffic.

Question 45

Question: What is a key benefit of using a NAT Gateway over a NAT instance in a VPC?

Choices:

A. Automatic scaling and high availability ✓

B. Direct support for load balancing

C. Support for custom routing rules

D. Lower cost for small-scale deployments

Explanation: NAT Gateway is managed and provides built-in availability and scaling characteristics compared with self-managed NAT instances.

Question 46

Question: Put the steps of DNS resolution for a domain hosted in a VPC in the correct order.

Choices:

- A. Resolver forwards the query to the authoritative nameserver
- B. Nameserver returns the IP address to the resolver
- C. Client sends a DNS query to the VPC DNS resolver
- D. Resolver returns the IP address to the client

Correct order:

1. C. Client sends a DNS query to the VPC DNS resolver ✓
2. A. Resolver forwards the query to the authoritative nameserver ✓
3. B. Nameserver returns the IP address to the resolver ✓
4. D. Resolver returns the IP address to the client ✓

Explanation: DNS resolution starts at the client resolver, queries authoritative sources, receives the IP, then returns it to the client.

Question 47

Question: What is the main challenge of managing state in a large-scale single-page application (SPA)?

Choices:

- A. Ensuring predictable state updates across components; solved with centralized state management ✓
- B. Rendering static content efficiently; solved with server-side rendering
- C. Handling CSS specificity; solved with CSS-in-JS
- D. Optimizing image loading; solved with lazy loading

Explanation: Large SPAs need predictable shared state updates; centralized state management helps coordinate components.

Question 48

Question: Which practices improve frontend performance for large-scale applications?

Choices:

A. Code splitting to reduce bundle size ✓

B. Disabling caching for resources

C. Increasing DOM mutations for dynamic updates

D. Using requestAnimationFrame for animations ✓

Explanation: Code splitting reduces initial payload, and requestAnimationFrame aligns animation work with browser rendering cycles.

Question 49

Question: How does the browser's Fetch API handle CORS compared to XMLHttpRequest?

Choices:

A. Fetch API only supports JSON; CORS is irrelevant

B. Fetch API uses cookies automatically; no CORS configuration needed

C. Fetch API bypasses CORS; no additional configuration needed

D. Fetch API enforces stricter CORS checks; requires explicit handling of preflight requests ✓

Explanation: Fetch follows browser CORS rules and may require preflight handling depending on method, headers, and credentials.

Question 50

Question: Arrange the steps to handle asynchronous data fetching in a web application in the correct order.

Choices:

A. Update the UI with the fetched data

B. Handle the response with Promise or async/await

C. Initiate an asynchronous API request using Fetch

Correct order:

1. C. Initiate an asynchronous API request using Fetch ✓

2. B. Handle the response with Promise or async/await ✓

3. A. Update the UI with the fetched data ✓

Explanation: The application first sends the request, waits for/handles the response, then updates the UI.

Question 51

Question: How does improper memory management in mobile apps lead to crashes, and what is a common mitigation?

Choices:

- A. Improper caching causes crashes; mitigated by clearing cache
- B. Excessive API calls cause crashes; mitigated by throttling
- C. Overloading CPU causes crashes; mitigated by reducing animations

D. Retaining unused objects causes memory exhaustion; mitigated by using weak references ✓

Explanation: Retained objects can leak memory until exhaustion; weak references help avoid unintended strong ownership cycles.

Question 52

Question: Arrange the steps to implement a secure authentication flow in a mobile app in the correct order.

Choices:

- A. Store the token securely in a protected storage
- B. Obtain user credentials securely via encrypted input
- C. Exchange credentials for a token using a secure API

Correct order:

1. B. Obtain user credentials securely via encrypted input ✓

2. C. Exchange credentials for a token using a secure API ✓

3. A. Store the token securely in a protected storage ✓

Explanation: Credentials are collected securely, exchanged over a secure API, and the resulting token is stored securely.

Question 53

Question: What is the impact of improper background task management in mobile applications?

Choices:

- A. It affects navigation; mitigated by proper routing
- B. It causes UI freezes; mitigated by async tasks

C. It drains battery and resources; mitigated by scheduling tasks with system APIs ✓

- D. It increases app size; mitigated by code splitting

Explanation: Uncontrolled background work wastes battery and resources; platform scheduling APIs coordinate background execution.

Question 54

Question: What is the main challenge of handling orientation changes in mobile applications?

Choices:

A. Optimizing animations; requires GPU acceleration

B. Preserving UI state across configuration changes; requires lifecycle-aware data handling ✓

C. Reducing app size; requires code splitting

D. Managing network requests; requires retry logic

Explanation: Orientation changes can recreate views; lifecycle-aware state management preserves user context.

Question 55

Question: During a code review, a senior engineer flags a piece of code that uses a series of nested `if-else if` statements to determine behavior based on an enum value. What clean code principle is most directly violated, and what refactoring pattern would be most appropriate to improve readability and maintainability?

Choices:

A. Violation: The Open/Closed Principle. Refactoring: Strategy Pattern. ✓

B. Violation: The Single Responsibility Principle. Refactoring: Facade Pattern.

C. Violation: Law of Demeter. Refactoring: Builder Pattern.

D. Violation: Don't Repeat Yourself (DRY). Refactoring: Template Method Pattern.

Explanation: Nested conditionals for behavior changes require modifying existing code for new cases. Strategy enables extension without modifying existing branching logic.

Question 56

Question: A senior developer advocates for extensive use of 'design by contract' (DbC) using assertions in a new module. What is the MOST significant 'clean code' benefit this provides, beyond simple error checking, and what is its primary limitation in production environments?

Choices:

A. Benefit: Reduces the need for defensive programming within functions. Limitation: Can introduce performance overhead if assertions are extensively used in performance-critical paths.

B. Benefit: Automates the generation of comprehensive unit tests for all public methods. Limitation: Increases the initial development time due to verbose contract specifications.

C. Benefit: Clearly defines expected behavior and obligations for callers and implementers. Limitation: Assertions are typically compiled out of production builds, losing runtime validation. ✓

D. Benefit: Provides robust error recovery mechanisms for unexpected runtime conditions. Limitation: Requires a dedicated runtime environment with assertion monitoring capabilities.

Explanation: DbC documents caller/implementer obligations and invariants, but assertions may be disabled in production depending on language/runtime settings.

Question 57

Question: A database administrator notices that a query with a WHERE clause on a non-indexed column is slow. Which approach would most effectively improve performance?

Choices:

- A. Partition the table by range
- B. Add a B+ tree index on the column ✓**
- C. Increase the transaction isolation level
- D. Use a stored procedure

Explanation: A B+ tree index lets the database locate matching rows without scanning the entire table.

Question 58

Question: Which Redis persistence options ensure no data loss in the event of a server crash, assuming proper configuration?

Choices:

- A. AOF with fsync set to everysec
- B. RDB snapshots taken every minute
- C. Combining AOF with always fsync and periodic RDB snapshots ✓**
- D. AOF with fsync set to always ✓**

Explanation: AOF always fsync minimizes acknowledged-write loss; combining it with snapshots adds recovery flexibility.

Question 59

Question: Which strategies can optimize Redis performance when handling high write throughput?

Choices:

- A. Increase the number of Redis instances without clustering
- B. Enable AOF with fsync set to always
- C. Use multi-key operations like MSET ✓**
- D. Use pipelining to batch commands ✓**

Explanation: Multi-key operations and pipelining reduce command round trips and improve throughput.

Question 60

Question: Which approach ensures consistent behavior in concurrent code without using locks?

Choices:

- A. Sleeping the thread before access
- B. Using global variables for shared state
- C. Running everything in a single thread

D. Immutable data structures and message-passing ✓

Explanation: Immutable data avoids shared mutation and message passing avoids direct shared-state coordination.

Question 61

Question: In a distributed system with multiple services and retries enabled, which scenario is most likely to cause a message to be processed more than once?

Choices:

- A. Retries are disabled and services process once only
- B. Producer sends only one message with no delay
- C. Retry mechanism lacks idempotency in the consumer ✓**
- D. The consumer acknowledges the message before processing

Explanation: Retries can deliver the same message more than once; idempotent consumers prevent duplicate side effects.

Question 62

Question: Which principle is most important for reducing code duplication and improving maintainability?

Choices:

- A. Write Everything Twice (WET)
- B. Don't Repeat Yourself (DRY) ✓**
- C. Premature optimization
- D. Global state usage

Explanation: DRY centralizes knowledge and reduces duplicated logic.

Question 63

Question: Which of the following best describes the purpose of using design patterns in software development?

Choices:

- A. They replace the need for understanding basic programming principles.

B. Design patterns are specific to a particular programming language and cannot be applied across different languages.

C. Design patterns are templates for solving common programming problems and can help developers create more efficient, reusable, and maintainable code. ✓

D. They are used solely for aesthetic purposes to make the code visually appealing.

Explanation: Design patterns provide reusable approaches to recurring design problems.

Question 64

Question: A team lead is integrating a legacy system through raw SQL queries. Which approach minimizes coupling and improves testability?

Choices:

A. Directly embed SQL queries within the application business logic.

B. Create a dedicated Anti-Corruption Layer (ACL). ✓

C. Use stored procedures only.

D. Map raw SQL results directly to domain objects without a boundary.

Explanation: An ACL isolates the domain model from legacy interfaces and makes integration more testable.

Question 65

Question: Why should commit messages be more descriptive than “Fix bug”?

Choices:

A. To simplify squashing commits.

B. To support git blame only.

C. To enable future git bisect operations to efficiently pinpoint regressions. ✓

D. To avoid writing tests.

Explanation: Descriptive commits make history useful for debugging, especially when bisecting regressions.

Question 66

Question: Which practice prevents merge conflicts in collaborative Git workflow?

Choices:

A. Never reviewing pull requests

B. Using only force pushes

C. Regularly pulling and rebasing from the main branch before pushing changes ✓

D. Committing large, infrequent changes

Explanation: Frequent synchronization reduces long-lived divergence and exposes conflicts earlier.

Question 67

Question: What are valid reasons for using asynchronous processing instead of synchronous APIs?

Choices:

A. To improve perceived responsiveness to users ✓

B. To enforce transactional guarantees automatically

C. To handle tasks that take a long time to complete ✓

D. To reduce memory consumption during heavy computation

Explanation: Async processing prevents long tasks from blocking user-facing flows and improves responsiveness.

Question 68

Question: Which is the most appropriate way to detect regression issues after deployment?

Choices:

A. Use automated integration and regression test suites ✓

B. Deploy to production without tests and observe behavior

C. Restart the system to clear bugs

D. Monitor logs manually for errors

Explanation: Automated regression tests repeatedly validate known behavior after changes.

Question 69

Question: What is the purpose of using a Git branching strategy in a collaborative project?

Choices:

A. Increase execution speed

B. Allow multiple developers to work on different features simultaneously without interfering with each other ✓

C. Ensure each commit is tagged with a version number

D. Automatically deploy changes to production

Explanation: Branches isolate work and allow parallel development while protecting the main line.

Question 70

Question: What is the primary purpose of a Factory Method in the Gang of Four design pattern?

Choices:

A. It provides a way to encapsulate object creation logic. ✓

B. It allows polymorphism without runtime type checks.

C. It enables adding features without modifying existing code.

D. It is used to implement Singleton in multithreaded environments.

Explanation: Factory Method separates creation logic from client code.

Question 71

Question: Which sequence correctly represents the Git workflow for contributing to a shared codebase?

Choices:

A. Commit and push changes to remote branch

B. Create a feature branch

C. Merge after review and testing

D. Open a pull request

Correct order:

1. B. Create a feature branch ✓

2. A. Commit and push changes to remote branch ✓

3. D. Open a pull request ✓

4. C. Merge after review and testing ✓

Explanation: A typical contribution flow is branch, commit/push, open PR, then merge after review.

Question 72

Question: Which of the following is the most subtle risk of using only code reviews for defect detection?

Choices:

A. Human reviewers may overlook subtle logic errors that automated tests would catch. ✓

B. Code reviews eliminate the need for testing.

C. Code reviews always guarantee 100% code coverage.

D. Code reviews always run faster than automated tests.

Explanation: Human review is valuable but cannot replace automated checks for subtle or repetitive cases.

Question 73

Question: Which type of test is best suited to validate how multiple services interact in production-like environments?

Choices:

A. Mutation testing

B. Static code analysis

C. End-to-end test ✓

D. Unit test

Explanation: E2E tests validate full flows across services and integrations.

Question 74

Question: Why does 100% code coverage not guarantee bug-free software?

Choices:

A. Coverage tools always miss private methods

B. Coverage is a measure of test speed

C. Coverage measures execution, not correctness or completeness of assertions ✓

D. Coverage is only relevant for integration tests

Explanation: Coverage shows code was executed, not that behavior was asserted correctly.

Question 75

Question: What is the purpose of a test coverage matrix?

Choices:

A. To map requirements to test cases and ensure all requirements are tested ✓

B. To automate deployment pipelines

C. To measure execution time of test cases

D. To list all bugs found during testing

Explanation: A coverage matrix links requirements to tests to verify traceability and completeness.

Question 76

Question: What is the subtle risk of relying only on integration tests in a microservices architecture?

Choices:

A. Integration tests guarantee 100% code coverage

B. Integration tests eliminate the need for monitoring

C. Defects in individual services may be masked by integration logic ✓

D. Integration tests always run faster than unit tests

Explanation: Integration tests can hide internal service defects behind orchestration behavior.

Question 77

Question: A feature passes locally but fails in CI. What is the most likely cause?

Choices:

A. Flaky test due to timing or external dependency ✓

B. Incorrect Git commit message format

C. Incompatible CPU architecture in the CI server

D. Missing semicolon in source code

Explanation: CI has different timing and dependency conditions, so flaky tests often appear there first.

Question 78

Question: What is the best approach when tests depend on external systems that intermittently return 500 errors?

Choices:

A. Disable the test to reduce pipeline failures

B. Increase CI server memory

C. Add retries to the test itself

D. Use a mock implementation that simulates failure conditions ✓

Explanation: Mocks isolate tests from unstable dependencies while preserving failure scenarios.

Question 79

Question: Which components should be included in a resilient financial system architecture?

Choices:

A. Retry with Exponential Backoff ✓

B. Circuit Breaker ✓

C. Load Balancer with Health Checks ✓

D. Static Content Delivery Network

E. Graceful Degradation ✓

F. Automated Failover to Disaster Recovery Site ✓

Explanation: These patterns improve availability, fault tolerance, and controlled recovery.

Question 80

Question: For a payment card processing system that must comply with PCI DSS, which security architecture component is most critical?

Choices:

A. Single Sign-On (SSO)

B. Intrusion Detection System (IDS)

C. Network segmentation with proper scoping ✓

D. Web Application Firewall (WAF)

Explanation: Segmentation reduces PCI scope and limits exposure of cardholder-data environments.

Question 81

Question: Which authentication mechanism provides the highest security for an investment banking portal?

Choices:

A. JWT with RSA signatures

B. Multi-factor authentication with hardware tokens and biometrics ✓

C. Basic authentication with HTTPS

D. OAuth 2.0 with PKCE

Explanation: Strong MFA reduces credential theft and account takeover risk.

Question 82

Question: Which strategies are valid for ensuring data consistency in distributed financial transaction systems?

Choices:

A. Circuit Breaker Pattern

B. Optimistic Concurrency Control ✓

C. ACID Transactions within Bounded Contexts ✓

D. Two-Phase Commit Protocol ✓

E. Eventual Consistency with Compensating Transactions ✓

F. Saga Pattern with Choreography ✓

Explanation: These strategies address consistency using locking/versioning, local ACID boundaries, distributed commit, compensation, or sagas.

Question 83

Question: Which deployment strategy minimizes downtime during updates?

Choices:

A. Rolling deployment

B. Blue-green deployment ✓

C. Shadow deployment

D. Canary deployment

Explanation: Blue-green deployment switches traffic between prepared environments for near-zero downtime.

Question 84

Question: Which partitioning strategy is best for data residency regulations?

Choices:

A. Feature-based partitioning

B. Hash-based partitioning

C. Round-robin partitioning

D. Geographic partitioning ✓

Explanation: Geographic partitioning keeps data in required jurisdictions.

Question 85

Question: Which monitoring approach detects API gateway security anomalies?

Choices:

- A. Request rate monitoring with threshold alerts
- B. Authentication failure counting with static thresholds
- C. Resource utilization tracking with historical comparison

D. Traffic pattern analysis with baseline deviation detection ✓

Explanation: Baseline deviation detection identifies abnormal traffic patterns beyond fixed thresholds.

Question 86

Question: Which monitoring approach identifies root cause in microservices performance issues?

Choices:

- A. VM-level resource monitoring with threshold alerts
- B. Synthetic transaction monitoring with baseline comparison
- C. Aggregated log analysis with pattern matching

D. Distributed tracing with context propagation ✓

Explanation: Distributed tracing follows a request across services and reveals latency sources.

Question 87

Question: What likely causes a table scan on queried columns?

Choices:

- A. The database is using a clustered index
- B. The query is using a materialized view
- C. The query is using a covering index

D. The table has no indexes on the queried columns ✓

Explanation: Without useful indexes, the database must scan rows to find matches.

Question 88

Question: Which approaches improve performance for expensive joins?

Choices:

- A. Increase the number of concurrent transactions
- B. Use appropriate indexes on join columns ✓**
- C. Denormalize the database to eliminate joins where justified ✓**
- D. Use a larger database page size

Explanation: Join indexes reduce lookup cost; selective denormalization can remove expensive joins.

Question 89

Question: Put database query optimization steps in correct order.

Choices:

- A. Parse the SQL query to validate syntax
- B. Estimate costs of execution plans
- C. Generate possible execution plans
- D. Select the optimal execution plan

Correct order:

- 1. A. Parse the SQL query to validate syntax ✓**
- 2. C. Generate possible execution plans ✓**
- 3. B. Estimate costs of execution plans ✓**
- 4. D. Select the optimal execution plan ✓**

Explanation: The optimizer parses SQL, generates alternatives, estimates costs, then selects the best plan.

Question 90

Question: Put Redis rate-limiting steps in correct order.

Choices:

- A. Reject or allow the request based on the counter value
- B. Set an expiration time on the key using EXPIRE
- C. Check if the counter exceeds the allowed limit
- D. Use INCR to increment a key with a user-specific identifier

Correct order:

1. D. Use INCR to increment a key with a user-specific identifier ✓
2. B. Set an expiration time on the key using EXPIRE ✓
3. C. Check if the counter exceeds the allowed limit ✓
4. A. Reject or allow the request based on the counter value ✓

Explanation: Increment first, set expiry for the window, check threshold, then allow or reject.

Question 91

Question: What is a key advantage of Redis Streams over pub/sub?

Choices:

- A. Streams have lower latency than pub/sub
- B. Streams support cross-cluster communication
- C. Streams allow dynamic channel creation

D. Streams provide message persistence ✓

Explanation: Redis Streams persist messages and support reliable consumption.

Question 92

Question: What is the primary benefit of a Redis Lua script for multiple commands?

Choices:

A. Ensures atomicity across multiple Redis commands ✓

- B. Improves performance by caching the script
- C. Enables cross-node operations in Redis Cluster
- D. Reduces memory usage compared to pipelines

Explanation: Lua scripts execute atomically within Redis.

Question 93

Question: How should Redis memory pressure from short-lived keys be mitigated?

Choices:

- A. Switch to AOF persistence
- B. Use Redis Cluster to distribute keys
- C. Enable the LRU eviction policy ✓**
- D. Increase the maxmemory limit

Explanation: LRU eviction removes least-recently-used cache entries when memory is constrained.

Question 94

Question: Which Dockerfile practices optimize image size?

Choices:

- A. Use multi-stage builds to reduce final image size ✓**
- B. Copy all project files before installing dependencies
- C. Minimize the number of layers by combining RUN commands ✓**
- D. Always use the latest base image version

Explanation: Multi-stage builds exclude build-only artifacts; fewer layers reduce image overhead.

Question 95

Question: What issue arises if .dockerignore is not properly configured?

Choices:

- A. The build cache is invalidated
- B. The build fails due to missing files
- C. Unnecessary files like .git or node_modules are copied, increasing image size ✓**
- D. The container cannot access the copied files

Explanation: A poor .dockerignore increases build context size and may leak unnecessary files into images.

Question 96

Question: In docker-compose.yml, what does depends_on ensure?

Choices:

- A. The specified service starts before the dependent service ✓**
- B. The services run in parallel with shared resources
- C. The dependent service restarts if the specified service fails
- D. The specified service is fully initialized before the dependent service starts

Explanation: depends_on controls startup order, not full readiness.

Question 97

Question: What is the difference between KStream and KTable?

Choices:

- A. KTable operations are always windowed
- B. KStream represents an append-only stream of records, while KTable represents a changelog stream where each key has the latest value ✓**
- C. Joining a KStream with a KTable is not possible
- D. KStream can only be read from a single topic

Explanation: KStream models event records; KTable models current state per key.

Question 98

Question: When does Kafka log compaction remove older records?

Choices:

- A. When the log segment containing the older record becomes inactive and the cleaner thread runs ✓**
- B. Immediately after a new message with the same key arrives
- C. Only when a message with the same key and a null value is produced
- D. After retention.ms has passed

Explanation: Compaction is asynchronous and operates on inactive segments.

Question 99

Question: What is the role of `__consumer_offsets` in Kafka?

Choices:

A. It stores the last committed offset for each partition consumed by a consumer group ✓

B. It tracks the high watermark for each partition

C. It stores messages consumed by all consumer groups

D. It is used by Kafka Streams to store intermediate state

Explanation: Kafka stores committed consumer offsets in the internal `__consumer_offsets` topic.

Question 100

Question: What is the primary purpose of SASL/SCRAM in Kafka?

Choices:

A. To provide a challenge-response mechanism for authenticating clients without sending passwords in cleartext ✓

B. Manage quotas

C. Authorize operations

D. Encrypt data in transit

Explanation: SASL/SCRAM is an authentication mechanism.

Question 101

Question: Which factors should be considered when configuring DNS TTL?

Choices:

A. Lower TTL for faster propagation of DNS changes ✓

B. Disabling DNS caching entirely

C. Using a single nameserver

D. Higher TTL to reduce DNS query costs ✓

Explanation: TTL trades off change propagation speed against cache efficiency and query cost.

Question 102

Question: Which infrastructure choices improve high availability across AZs?

Choices:

A. Application Load Balancer with cross-zone load balancing ✓

B. Multiple subnets across different Availability Zones ✓

C. A single NAT instance for outbound traffic

D. A single security group for all instances

Explanation: Cross-zone load balancing and multi-AZ subnets improve availability.

Question 103

Question: In a VPC, what is the primary purpose of a security group?

Choices:

A. To assign IP addresses to instances

B. To act as a virtual firewall for controlling inbound and outbound traffic ✓

C. To encrypt data in transit between instances

D. To route traffic between subnets

Explanation: Security groups control inbound and outbound traffic for resources.

Question 104

Question: Why is CSP critical for frontend security?

Choices:

A. It encrypts HTTP requests

B. It optimizes resource caching

C. It manages user authentication

D. It restricts resource loading to trusted sources, mitigating XSS attacks ✓

Explanation: CSP reduces XSS risk by limiting where scripts and resources can load from.

Question 105

Question: How does Critical Rendering Path impact page load performance?

Choices:

- A. It manages event loop scheduling
- B. It determines the order of resource processing for initial render; optimizing it reduces time to first paint ✓**
- C. It handles service worker registration
- D. It controls caching strategies

Explanation: Optimizing the rendering path improves how quickly visible content appears.

Question 106

Question: Why is lazy loading critical for large-scale web applications?

Choices:

- A. It simplifies state management
- B. It caches resources for offline use
- C. It encrypts resources
- D. It defers loading of non-critical resources, reducing initial page load time ✓**

Explanation: Lazy loading delays non-critical work until needed.

Question 107

Question: What causes layout thrashing and how can it be mitigated?

Choices:

- A. Excessive CSS animations
- B. Large JavaScript bundles
- C. Forced synchronous layout due to alternating reads and writes; mitigated by batching DOM updates ✓**
- D. Unoptimized images

Explanation: Batching DOM reads and writes avoids repeated forced reflow.

Question 108

Question: What is the primary benefit of reactive architecture in mobile apps?

Choices:

- A. Simplifies database queries
- B. Encrypts data
- C. Reduces app size

D. Ensures UI updates are synchronized with data changes, improving responsiveness ✓

Explanation: Reactive architecture propagates data changes to UI consistently.

Question 109

Question: Which technique improves mobile app scalability under high user load?

Choices:

- A. Disabling background sync
- B. Using efficient data structures for local storage

C. Implementing load balancing for backend services ✓

- D. Increasing image resolution

Explanation: High user load is usually handled by scaling backend capacity with load balancing.

Question 110

Question: Which practices improve mobile reliability during network disruptions?

Choices:

A. Using optimistic updates with rollback on failure ✓

- B. Increasing synchronous API calls

C. Implementing retry logic with exponential backoff ✓

- D. Disabling local caching

Explanation: Optimistic updates improve perceived responsiveness, and exponential backoff handles transient failures without overload.

Question 111

Question: Order the steps for performing a database schema migration in a zero-downtime deployment strategy.

Choices:

- A. Apply new schema changes (e.g., add new column)
- B. Deploy application code that supports both old and new schema
- C. Gradually shift traffic to new code using new schema
- D. Remove old schema and code once migration is verified

Correct order:

1. A. Apply new schema changes (e.g., add new column) ✓
2. B. Deploy application code that supports both old and new schema ✓
3. C. Gradually shift traffic to new code using new schema ✓
4. D. Remove old schema and code once migration is verified ✓

Explanation: Zero-downtime schema migration commonly uses an expand-and-contract approach: add backward-compatible schema first, deploy compatible code, shift traffic, then remove old schema only after verification.

Question 112

Question: Which approach best improves code readability and maintainability?

Choices:

- A. Relying on comments instead of clear code
- B. Using descriptive variable and function names ✓
- C. Writing all logic in a single file
- D. Using abbreviations to shorten code

Explanation: Descriptive names make intent clear at the point of use and reduce the need for explanatory comments.

Question 113

Question: Which of the following are advantages of Infrastructure as Code (IaC)?

Choices:

- A. Reduces human error in manual deployments ✓
- B. Allows for configuration drift between environments
- C. Improves reproducibility of environments ✓
- D. Makes it harder to version control infrastructure

Explanation: IaC makes infrastructure definitions repeatable and versionable, reducing manual deployment mistakes and improving environment consistency.

Question 114

Question: Which design pattern is most suitable for decoupling object creation from usage?

Choices:

A. Decorator pattern

B. Factory pattern ✓

C. Observer pattern

D. Singleton pattern

Explanation: A Factory pattern encapsulates object creation so client code depends on abstractions rather than concrete construction details.

Question 115

Question: Which design principle most effectively reduces the risk of ripple effects when modifying code in a large codebase?

Choices:

A. Code duplication

B. Loose coupling ✓

C. Global variables

D. Deep inheritance

Explanation: Loose coupling reduces unnecessary dependencies, so changes in one module are less likely to cascade through unrelated modules.

Question 116

Question: Which of the following is the most subtle risk when relying solely on UI testing for application quality?

Choices:

A. UI tests guarantee 100% code coverage.

B. UI tests eliminate the need for manual testing.

C. UI tests always run faster than unit tests.

D. UI tests may miss critical backend logic errors that are not exposed through the interface. ✓

Explanation: UI tests validate visible flows, but backend rules and edge cases may remain untested if they are not surfaced through the UI.

Question 117

Question: Which of the following is the most subtle risk of using only positive test cases in API testing?

Choices:

- A. Positive tests eliminate the need for documentation.
- B. Positive tests guarantee 100% code coverage.
- C. Positive tests always run slower.

D. Negative scenarios may go untested, leading to unhandled errors in production. ✓

Explanation: Positive-only API tests validate expected success paths but miss invalid inputs, authorization failures, malformed payloads, and other production error cases.

Question 118

Question: Which scenario best demonstrates the limitation of code coverage as a sole quality metric?

Choices:

A. Tests execute every line but do not assert any outcomes. ✓

- B. Tests are slow to execute.
- C. Tests are written for only public methods.
- D. Tests use mocks for all dependencies.

Explanation: Executing every line does not prove correctness if tests do not assert expected outcomes.

Question 119

Question: A bank is implementing a predictive maintenance strategy for its core banking system. Which data source would provide the most valuable input for predicting potential system failures?

Choices:

- A. Real-time performance telemetry with anomaly detection
- B. User feedback and reported issues

C. Historical incident data correlated with system metrics ✓

- D. Log analysis with pattern recognition

Explanation: Historical incidents correlated with system metrics provide labeled patterns that are valuable for predicting future failures.

Question 120

Question: When monitoring a critical financial application across multiple cloud regions, which metric would provide the most accurate insight into global system health?

Choices:

- A. Average response time across all regions
- B. Error rate percentiles by geographic region
- C. Infrastructure utilization across availability zones

D. Service level indicator (SLI) compliance by service and region ✓

Explanation: SLI compliance by service and region captures user-facing reliability against defined service objectives across the global footprint.

Question 121

Question: In a relational database, a table is in Third Normal Form (3NF) but not in Boyce-Codd Normal Form (BCNF). What is the most likely issue?

Choices:

- A. The table has a non-prime attribute that is transitively dependent on the primary key
- B. The table has a non-prime attribute that is functionally dependent on a subset of a candidate key
- C. A non-key attribute is functionally dependent on another non-key attribute

D. A non-candidate key attribute is functionally dependent on a subset of a candidate key ✓

Explanation: BCNF is stricter than 3NF and requires every determinant to be a candidate key; violations often involve dependencies where a determinant is not a candidate key.

Question 122

Question: Arrange the steps of the ACID transaction process in the correct order.

Choices:

- A. Begin the transaction
- B. Execute operations within the transaction
- C. Rollback if any operation fails
- D. Commit the transaction

Correct order:

1. A. Begin the transaction ✓
2. B. Execute operations within the transaction ✓
3. C. Rollback if any operation fails ✓
4. D. Commit the transaction ✓

Explanation: A transaction begins, operations execute, failures trigger rollback, and successful completion commits the transaction.

Question 123

Question: A Redis instance is experiencing high latency during peak traffic due to frequent access to a large sorted set. Which approach would most effectively reduce latency while maintaining functionality?

Choices:

- A. Switch to a Redis hash instead of a sorted set
- B. Enable compression for the sorted set data
- C. Use Redis Cluster to distribute the sorted set across multiple nodes ✓
- D. Increase the memory allocation for the Redis instance

Explanation: Redis Cluster can distribute load and data across nodes, reducing hot-spot pressure for large high-traffic workloads.

Question 124

Question: Which Redis commands are most suitable for implementing a distributed lock?

Choices:

A. EXPIRE ✓

B. GETSET

C. SETNX ✓

D. DEL ✓

Explanation: SETNX can acquire a lock if absent, EXPIRE prevents permanent stale locks, and DEL releases the lock when work is complete.

Question 125

Question: Arrange the steps to create an optimized Dockerfile for a Node.js application, maximizing build cache efficiency and minimizing image size.

Choices:

A. Use a specific base image version (e.g., FROM node:18.16.0-alpine)

B. Copy only dependency definition files (e.g., package.json and package-lock.json)

C. Run dependency installation (e.g., npm ci)

D. Copy the application source code

E. Set the CMD instruction in exec form (e.g., CMD ["npm", "start"])

Correct order:

1. A. Use a specific base image version (e.g., FROM node:18.16.0-alpine) ✓

2. B. Copy only dependency definition files (e.g., package.json and package-lock.json) ✓

3. C. Run dependency installation (e.g., npm ci) ✓

4. D. Copy the application source code ✓

5. E. Set the CMD instruction in exec form (e.g., CMD ["npm", "start"]) ✓

Explanation: Copying dependency files before source code lets Docker reuse dependency-install layers when application code changes.

Question 126

Question: Which of the following can cause a Docker build to fail?

Choices:

- A. An invalid command in a RUN instruction ✓**
- B. A missing file referenced in a COPY instruction ✓**
- C. Specifying multiple CMD instructions
- D. Using a non-existent base image in FROM ✓**

Explanation: Invalid RUN commands, missing COPY sources, and non-existent base images are build-time failures; multiple CMD instructions are allowed but only the last one takes effect.

Question 127

Question: In a Kafka cluster with a topic having a replication factor of 3 and min.insync.replicas set to 2, what is the maximum number of broker failures the cluster can tolerate for that topic's partitions while still being able to produce messages with acks=all?

Choices:

- A. 2
- B. 0
- C. 3
- D. 1 ✓**

Explanation: With replication factor 3 and min.insync.replicas 2, one broker can fail while still leaving two in-sync replicas for acks=all writes.

Question 128

Question: Which of the following statements about Kafka's exactly-once semantics (EOS) is most accurate?

Choices:

- A. EOS in Kafka is primarily achieved through a combination of idempotent producers, atomic transactions, and cooperative consumer rebalancing. ✓**
- B. EOS is achieved solely by enabling idempotent producers (enable.idempotence=true).
- C. Enabling EOS has no significant performance overhead.
- D. EOS in Kafka guarantees that a message is delivered and processed exactly once from end-to-end, including external systems.

Explanation: Kafka EOS relies on idempotent production and transactions within Kafka processing boundaries; it is not simply a single producer flag nor a guarantee across all external systems.

Question 129

Question: What happens when a Kafka producer sends a message to a partition whose leader is currently unavailable?

Choices:

- A. The producer will immediately receive a `NotLeaderForPartitionException`.
- B. The producer will send the message to a follower replica instead.
- C. The message is immediately lost.

D. The producer will buffer the message and retry sending after its `metadata.max.age.ms` expires, forcing a metadata refresh to find the new leader. ✓

Explanation: Kafka producers rely on metadata and retries to discover the new leader rather than sending writes to follower replicas.

Question 130

Question: What happens when a DNS query fails to resolve due to a missing record in an authoritative nameserver?

Choices:

- A. The resolver retries the query with a different protocol
- B. The client receives an `NXDOMAIN` response ✓**
- C. The query is cached indefinitely
- D. The query is forwarded to a root nameserver

Explanation: `NXDOMAIN` indicates that the requested domain name does not exist in the authoritative DNS data.

Question 131

Question: Arrange the steps to optimize the Critical Rendering Path in the correct order.

Choices:

- A. Minimize render-blocking CSS and JavaScript
- B. Optimize critical CSS for above-the-fold content
- C. Defer non-critical JavaScript execution

Correct order:

- 1. A. Minimize render-blocking CSS and JavaScript ✓**
- 2. B. Optimize critical CSS for above-the-fold content ✓**
- 3. C. Defer non-critical JavaScript execution ✓**

Explanation: Critical rendering path optimization first reduces blocking resources, prioritizes above-the-fold CSS, and defers non-critical JavaScript.

Question 132

Question: What is the role of the IntersectionObserver API in modern frontend development?

Choices:

- A. It handles touch events, improving mobile UX
- B. It monitors element visibility in the viewport, enabling lazy loading and animations ✓**
- C. It manages WebSocket connections, enabling real-time updates
- D. It optimizes CSS rendering, reducing reflows

Explanation: IntersectionObserver notifies code when elements enter or leave the viewport, which supports lazy loading and scroll-triggered behavior.

Question 133

Question: Put the steps to implement lazy loading for images in the correct order.

Choices:

- A. Add the loading='lazy' attribute to img tags
- B. Use IntersectionObserver to detect image visibility
- C. Load image sources when they enter the viewport

Correct order:

- 1. A. Add the loading='lazy' attribute to img tags ✓**
- 2. B. Use IntersectionObserver to detect image visibility ✓**
- 3. C. Load image sources when they enter the viewport ✓**

Explanation: Native lazy loading can be enabled directly, while IntersectionObserver can detect visibility and trigger loading when needed.

Question 134

Question: What is the primary benefit of using a virtual DOM in frontend applications?

Choices:

- A. It minimizes direct DOM manipulations, improving performance ✓**
- B. It handles HTTP requests, simplifying API calls
- C. It encrypts client-side data, enhancing security
- D. It enables server-side rendering, improving SEO

Explanation: A virtual DOM batches and reconciles UI changes to reduce expensive direct DOM operations.

Question 135

Question: Arrange the steps to optimize mobile app startup time in the correct order.

Choices:

- A. Initialize critical UI components
- B. Defer non-critical resource loading
- C. Perform background tasks after UI rendering

Correct order:

1. A. Initialize critical UI components ✓
2. B. Defer non-critical resource loading ✓
3. C. Perform background tasks after UI rendering ✓

Explanation: Startup should prioritize critical UI first, delay non-essential work, and run background tasks after initial rendering.

Question 136

Question: Why is deep linking critical for mobile app user experience, and what is a key challenge in implementing it?

Choices:

- A. It manages push notifications; challenge is user permissions
- B. It optimizes app startup time; challenge is resource loading
- C. It enables direct navigation to app content; challenge is handling platform-specific URI schemes ✓
- D. It improves app security; challenge is encryption

Explanation: Deep links take users directly to relevant in-app content, but implementations must handle platform-specific URI and routing behavior.